

MODULE 0

DURATION: 3 DAYS

Induction + Orientation

STUDY NOTES

Understand the GenAI & Agentic AI landscape, the tools you will use, and how this program maps to your career goals.

WHAT'S INSIDE

- ▶ Topic 1: Overview of AI, Generative AI and Agentic AI
- ▶ Topic 2: Career Paths and Role Mapping
- ▶ Topic 3: LinkedIn Profile Optimization for AI & Tech Hiring
- ▶ Topic 4: Using Google Colab for Hands-On Learning
- ▶ Topic 5: Learning Workflow, Tools and Program Onboarding

Contents

Topic 1: Overview of AI, Generative AI and Agentic AI

3

Artificial Intelligence (AI)

3

Generative AI (GenAI)

4

Agentic AI

5

Topic 2: Career Paths and Role Mapping

6

AI Career Roles

6

Mapping Your Current Role

6

Topic 3: LinkedIn Profile Optimization for AI & Tech Hiring

8

Profile Essentials

8

Topic 4: Using Google Colab for Hands-On Learning

10

Google Colab Basics

10

Topic 5: Learning Workflow, Tools and Program Onboarding

12

Your Developer Toolkit

12

Weekly Learning Loop

13

How to use these notes

Every topic in these notes follows the same pattern: a crisp **definition**, an **easy-to-remember** analogy, a detailed explanation, **syntax and code** where it applies, a **real-life example**, **how** and **where** to use it, and its **limitations**. Coloured boxes always mean the same thing, so you can scan quickly during revision.

PRO TIP

Read a topic once, run every code block yourself (in Colab or VS Code), then close the notes and explain the "Easy to remember" box out loud. If you can explain it simply, you have learned it.

NOTE

AI libraries and model names change fast. Code in these notes uses widely used, current-style APIs, but always check the library version you have installed (`pip show <package>`) and the official docs if a line errors.

TOPIC 1

Overview of AI, Generative AI and Agentic AI

■ Artificial Intelligence (AI)

DEFINITION

Artificial Intelligence is the field of building computer systems that perform tasks which normally need human intelligence — recognising patterns, understanding language, making predictions and taking decisions.

EASY TO REMEMBER

AI is like a trained intern. You don't hand an intern a rule for every situation; you show examples, they learn the pattern, and slowly they handle new cases on their own.

Older AI was mostly **rule-based**: programmers wrote IF-THEN rules (called expert systems). Modern AI is mostly **learning-based**: the system learns the rules from data. The AI family is nested like Russian dolls — each layer is a special case of the one above it.

Layer	What it means	Everyday example
Artificial Intelligence	Any technique that makes machines act intelligently	Chess engine, route planner
Machine Learning (ML)	Systems that learn patterns from data instead of hand-written rules	Spam filter, loan-default prediction
Deep Learning (DL)	ML using many-layered neural networks	Face unlock, speech-to-text
Generative AI	Deep-learning models that create new content	ChatGPT, Claude, image generators
Agentic AI	GenAI models that plan and take actions with tools to reach a goal	AI that researches, fills forms and books a ticket

REAL-LIFE EXAMPLE

When Google Maps predicts "18 min to office", ML models combine live traffic, historical patterns and road data. Nobody wrote a rule for your exact route at 9:12 am — the model learned it.

WHERE TO USE IT

- Predictions (sales, demand, risk), recognition (images, speech), recommendations (products, videos), and automating repetitive decisions.

LIMITATIONS

- Needs large amounts of good-quality data, and learns the biases hidden in that data.
- Many models are "black boxes" — it is hard to explain why a particular decision was made.
- Performs poorly on situations that are very different from its training data.

■ Generative AI (GenAI)

DEFINITION

Generative AI is a type of AI that **creates new content** — text, images, code, audio or video — by learning the patterns in a huge amount of existing data and producing something similar but new.

EASY TO REMEMBER

Predictive AI is a **judge**: it looks at something and gives a verdict ("spam / not spam").

Generative AI is an **author**: it writes something new ("draft a polite reply to this email").

Large Language Models (LLMs) such as GPT, Claude, Gemini and Llama are the most popular GenAI models. They are trained to predict the **next token** (a piece of a word) given the text before it. Doing this billions of times on internet-scale text gives them broad knowledge of language, facts, reasoning patterns and code. Image models (Stable Diffusion, FLUX) learn to turn random noise into pictures that match a text description.

Example — your first GenAI call (Python, OpenAI SDK)

```
# pip install openai          (set the OPENAI_API_KEY environment variable first)
from openai import OpenAI
client = OpenAI()
response = client.chat.completions.create(
    model="gpt-4o-mini",        # model names change often; check the provider's model list
    messages=[{"role": "user", "content": "Explain GenAI to a 10-year-old in 2 lines."}],
)
print(response.choices[0].message.content)
```

REAL-LIFE EXAMPLE

A bank relationship manager pastes a 30-page annual report into an LLM and gets a one-page summary of revenue, risks and key ratios in seconds — work that used to take an analyst an afternoon.

WHERE TO USE IT

- Drafting emails, reports and marketing copy; summarising documents; generating and explaining code; chatbots and knowledge assistants; creating images and designs.

LIMITATIONS

- **Hallucination**: it can produce confident but wrong answers.
- It has a knowledge cut-off and no access to your private data unless you provide it (RAG, covered in Module 3, solves this).
- Cost, latency, data privacy and copyright are real concerns in enterprise use.

■ Agentic AI

DEFINITION

Agentic AI refers to AI systems that **pursue a goal on their own**: they break the goal into steps, decide which tools to use (search, databases, APIs, code), act, observe the results and adjust — with limited human supervision.

EASY TO REMEMBER

A chatbot **tells** you how to book a train ticket. An agent **books** it: checks your calendar, compares trains, fills the booking form, and asks "Shall I pay ₹1,240?" before the final step.

An agent is usually built from four parts: an **LLM brain** (reasoning and planning), **tools** (functions it can call), **memory** (what has happened so far and what it knows about you) and **guardrails** (limits and human approvals). It runs in a loop: **Perceive → Plan → Act → Observe → Reflect → repeat** until the goal is reached.

Pseudo-code — the agent loop

```
goal = "Find the 3 cheapest flights Hyderabad-Delhi next Friday and email them to me"
memory = []
while not goal_reached(memory):
    step = llm.decide_next_step(goal, memory)      # THINK: what should I do next?
    if step.needs_human_approval:
        ask_user_to_confirm(step)                 # GUARDRAIL before risky actions
    result = run_tool(step.tool, step.arguments)   # ACT: search, email, ...
    memory.append((step, result))                  # OBSERVE & REMEMBER
```

Aspect	Traditional ML	Generative AI	Agentic AI
Main job	Predict / classify	Create content	Achieve a goal
Output	A label or number	Text, image, code	Completed actions + results
Interaction	One prediction	One prompt → one answer	Multi-step loop with tools
Example	Fraud score = 0.87	Draft of a complaint reply	Investigates, drafts and files the case

REAL-LIFE EXAMPLE

An IT-support agent receives "VPN not working", checks the user's account status through an API, finds an expired certificate, re-issues it and closes the ticket — escalating to a human only if the fix fails.

WHERE TO USE IT

- Multi-step workflows: customer-support resolution, research and report generation, data analysis, DevOps incident handling, and back-office automation (KYC, claims, invoices).

LIMITATIONS

- Errors compound across steps; one wrong tool call can derail the whole task.
- Harder to test, monitor and secure (prompt injection, unsafe actions).
- Higher cost and latency because of many LLM calls; risky actions need a human in the loop.

TOPIC 2

Career Paths and Role Mapping

AI Career Roles

DEFINITION

Role mapping means matching your current skills and domain experience to a specific AI job role, then identifying the gap you need to close.

EASY TO REMEMBER

Think of it like Google Maps for your career: you need your **current location** (today's skills), your **destination** (target role) and the **route** (modules, projects, electives).

AI job titles vary between companies, but the work falls into a few clear families. Use the table to see which modules of this program matter most for each role.

Role	What they do	Key skills	Most relevant modules
Data Analyst	Turns data into dashboards and insights	SQL, Excel, Python, Pandas, charts	M1
Data Scientist	Builds predictive models and runs experiments	Statistics, ML, Python, storytelling	M1, M2
ML Engineer	Takes ML models into production	scikit-learn/PyTorch, APIs, MLOps	M2, M5
GenAI / LLM Engineer	Builds LLM apps: prompts, RAG, fine-tuning	LLM APIs, LangChain, vector DBs, evaluation	M3, M5
AI Agent Engineer	Designs multi-step, tool-using agents	LangGraph, CrewAI, AutoGen, MCP	M4
LLMOps / MLOps Engineer	Deploys, monitors and scales AI systems	Docker, Kubernetes, cloud, observability	M5
AI Product Manager	Decides what AI to build and measures value	Use-case discovery, ROI, UX, AI literacy	M3, M6, Elective F
AI Solutions Architect	Designs end-to-end enterprise AI systems	Azure/AWS, security, integration	M3–M5, Elective G
Responsible-AI / Governance Specialist	Keeps AI safe, fair and compliant	Regulation, risk, audits, guardrails	M6

Mapping Your Current Role

Your existing domain knowledge is an advantage, not a burden. A banker who understands KYC builds a far better KYC agent than a coder who has never seen a KYC form.

If you are a...	Natural next AI role	Suggested electives
Software developer	GenAI engineer / AI agent engineer	G (Developers), E (Cloud)

If you are a...	Natural next AI role	Suggested electives
Tester / QA engineer	AI QA engineer, LLM evaluation engineer	H (QA)
DevOps / cloud engineer	LLMOps engineer	E (Cloud & DevOps)
Banking / insurance professional	AI analyst or product owner in BFSI	A (BFSI), F
Doctor, pharmacist, healthcare staff	Clinical AI specialist	D (Healthcare)
Manager / team lead	AI product manager, AI transformation lead	F (Managers)
Plant / supply-chain engineer	Industrial AI specialist	C (Manufacturing)
Retail / e-commerce professional	AI for growth and merchandising	B (Retail)

HOW TO USE IT

1. List your current skills (technical + domain) honestly.
2. Pick one target role from the table and read 10 real job descriptions for it.
3. Mark the skills you already have, and the gaps.
4. Close the gaps with modules and build 3–4 portfolio projects in **your** domain.
5. Update LinkedIn and GitHub as you go (next topic).

REAL-LIFE EXAMPLE

A QA engineer with 6 years of experience targets "AI QA Engineer". She already knows test design and Jira, so her gap is LLM APIs and evaluation. She picks Elective H and builds a test-case generator from user stories — a project interviewers instantly relate to.

LIMITATIONS

- Titles differ widely ("AI Engineer" can mean anything from prompt work to model training), so always read the job description, not just the title.
- Role mapping is a starting plan; the market changes, so revisit it every few months.

TOPIC 3

LinkedIn Profile Optimization for AI & Tech Hiring

■ Profile Essentials

DEFINITION

LinkedIn optimization means structuring your profile — headline, About, experience, skills and projects — so that recruiters searching for AI skills can find you and quickly trust what you can do.

EASY TO REMEMBER

Your LinkedIn profile is a **shop window**. The headline is the signboard, the About section is the shopkeeper's pitch, and Featured projects are the products on display. Recruiters walk past in two seconds — make the window clear.

Recruiters search with keywords (e.g. "RAG", "LangChain", "LLM", "Python") and filters (location, years of experience). If those words are missing from your profile, you are invisible, no matter how skilled you are.

- **Headline** — Formula: *Current role* → *Target role* | 3-4 key skills | Domain.
- **About** — 4 short paragraphs: who you are, what you build, proof (projects/metrics), what you are looking for.
- **Experience** — Bullet achievements with numbers: "Built X using Y, which reduced Z by N%".
- **Featured** — Pin 2-3 best projects: GitHub repo, demo video, or a blog post.
- **Skills** — Add relevant skills (Python, LLMs, RAG, LangGraph, FastAPI, Docker, Azure OpenAI...) and get endorsements.
- **Activity** — Post once a week about what you built or learned; it signals active learning.

Example — headline and About section (text template)

HEADLINE:

QA Engineer → GenAI QA Engineer | LLM Evaluation · RAG · LangChain · Pytest | BFSI

ABOUT:

I'm a QA engineer with 6 years in banking software, now building AI systems that make testing faster and smarter.

Recent projects:

- Test-case generator that turns Jira user stories into prioritised test cases (LangChain + OpenAI)
- RAG bot over 400 pages of test documentation – answers in < 3 s with source citations

Tools: Python, LangChain, LangGraph, ChromaDB, FastAPI, Docker, Jira API

Open to: GenAI QA / LLM Evaluation roles in Hyderabad or remote.

REAL-LIFE EXAMPLE

A recruiter at a fintech searches "RAG AND LangChain AND Hyderabad". Two candidates have the same skills, but only one lists those words in her headline and skills. Only she appears in the results.

HOW TO USE IT

- Rewrite your headline first (biggest impact), then About, then add one Featured project each time you finish a project in this program.

WHERE TO USE IT

- Job search, freelancing, internal transfers, and building credibility before you change roles.

LIMITATIONS

- A great profile gets you noticed, but projects and interviews get you hired — every claim must be backed by real work.
- Avoid keyword stuffing; it looks spammy and recruiters notice.

TOPIC 4

Using Google Colab for Hands-On Learning

Google Colab Basics

DEFINITION

Google Colab (Colaboratory) is a free, browser-based **Jupyter notebook** service from Google that runs Python on cloud machines, with optional GPU/TPU access — no installation required.

EASY TO REMEMBER

Colab is **Google Docs for code**: open it in a browser, write, run, share a link — and Google lends you the computer (even a GPU) for free.

A notebook is made of **cells**. A **code cell** runs Python; a **text (Markdown) cell** holds notes. Variables stay in memory across cells until the runtime restarts. Lines starting with **!** run shell commands (for example **!pip install**), and **%** lines are notebook "magic" commands (for example **%timeit**).

Syntax — everyday Colab commands

```
!pip install -q openai langchain chromadb      # install packages (-q = quiet)
!nvidia-smi                                    # check which GPU you got
import torch
print(torch.cuda.is_available())                # True if a GPU runtime is active
from google.colab import drive
drive.mount('/content/drive')                  # access your Google Drive files
from google.colab import userdata
api_key = userdata.get('OPENAI_API_KEY')        # read a key stored in Colab "Secrets" (key panel)
%timeit sum(range(1_000_000))                  # magic command: time a line
```

Shortcut	Action
Shift + Enter	Run cell and move to the next one
Ctrl + M, then B / A	Insert a code cell below / above
Ctrl + M, then M / Y	Convert cell to text / code
Ctrl + M, then D	Delete cell
Runtime → Change runtime type	Switch to GPU (e.g. T4) or back to CPU

REAL-LIFE EXAMPLE

You want to fine-tune a small sentiment model, but your laptop has no GPU. In Colab you switch the runtime to a T4 GPU, mount Drive to load the dataset, train for 20 minutes, and save the model back to Drive.

HOW TO USE IT

- Use Colab for experiments, learning, and quick demos; save notebooks to GitHub or Drive.
- Store API keys in **Secrets**, never hard-code them in a shared notebook.

WHERE TO USE IT

- Learning, prototyping ML/GenAI ideas, GPU experiments, and sharing reproducible demos with mentors.

LIMITATIONS

- Sessions disconnect after inactivity and have a maximum lifetime; files in `/content` are wiped when the runtime resets — save important work to Drive.
- Free-tier GPUs are not guaranteed and have usage limits.
- Not suitable for hosting production apps or long-running services.

TOPIC 5

Learning Workflow, Tools and Program Onboarding

■ Your Developer Toolkit

DEFINITION

A learning workflow is the repeatable routine and set of tools you use to learn a concept, practise it in code, and turn it into a portfolio piece.

EASY TO REMEMBER

Think **gym routine**: warm-up (concept), workout (code-along), extra set (mini-project), and progress photo (GitHub commit + LinkedIn post). Skip the routine and gains disappear.

Tool	Why you need it
Python 3.11+	The main language of this program
VS Code	Code editor with Python, Jupyter and Docker extensions
Colab / Jupyter	Notebooks for experiments
Git + GitHub	Version control and public portfolio
Virtual environments (venv / conda / uv)	Keep each project's packages separate
.env files + python-dotenv	Keep API keys out of your code
Docker	Package apps so they run the same everywhere (Module 5)
Postman / curl	Test the APIs you build

Syntax — set up a new project (terminal)

```
mkdir genai-projects && cd genai-projects
python -m venv .venv
source .venv/bin/activate           # Windows: .venv\Scripts\activate
pip install openai python-dotenv pandas
pip freeze > requirements.txt       # record exact versions for others
echo "OPENAI_API_KEY=sk-..." > .env
echo ".env" >> .gitignore           # never commit secrets
```

Example — load secrets safely in Python

```
import os
from dotenv import load_dotenv
load_dotenv()                       # reads variables from .env
api_key = os.getenv("OPENAI_API_KEY")
if not api_key:
    raise RuntimeError("OPENAI_API_KEY is missing – check your .env file")
```

Syntax — push your work to GitHub

```
git init
git add .
git commit -m "Project 1: simple calculator"
git branch -M main
git remote add origin https://github.com/<your-username>/genai-projects.git
git push -u origin main
```

■ Weekly Learning Loop

STEP-BY-STEP

1. **Learn** the concept (live class + these notes).
2. **Code along** — retype the examples, don't copy-paste.
3. **Break it** — change parameters and inputs, observe what happens.
4. **Mini-project** — apply it to a problem from your own domain.
5. **Ship** — push to GitHub with a clear README (problem, approach, how to run, screenshots).
6. **Share** — a short LinkedIn post: what you built and one thing you learned.

REAL-LIFE EXAMPLE

Learners who push one small project every week finish the program with 30+ public repositories — a portfolio that speaks louder than any certificate.

LIMITATIONS

- Tools are only enablers; consistency matters more than the perfect setup. Don't spend a week configuring your editor.